

Eötvös Loránd University

Faculty of Informatics

Cloud-Native Deployment of the AIDA Learning Platform

Semester Project Report

AIDA (AI-driven Instructional Design Assistant)

Course: DevOps and Cloud engineering on Azure

Student: Jahonoro Tojjeva

Neptun Code: RVKIPF

Academic Year: 2025/2026

Submission Date: June 2026

Budapest, Hungary

Contents

1	Introduction	2
2	Application Overview	2
3	System Architecture	5
4	Cloud Infrastructure Design	6
5	Infrastructure as Code with Terraform	9
6	Containerization and Application Deployment	11
7	Database Backup and Recovery	12
8	CI/CD Pipeline Implementation	13
9	Security Considerations	17
10	Challenges and Solutions	17
11	Challenges and Solutions	18

1 Introduction

Cloud-native technologies have become an essential part of modern software development. Instead of deploying applications manually on physical servers, organizations increasingly rely on cloud platforms, Infrastructure as Code (IaC), containerization, and Continuous Integration/Continuous Deployment (CI/CD) pipelines to automate the deployment and maintenance process. These approaches improve scalability, reliability, and development efficiency while reducing manual operational work.

The goal of this project was to design, deploy, and automate a complete cloud-based application using Microsoft Azure. The selected application for this semester task is **AIDA (AI-driven Instructional Design Assistant)**, an educational platform that combines online learning functionality with artificial intelligence tools. The platform allows users to register accounts, enroll in courses, complete lessons, generate AI-powered quizzes, track their learning progress, and obtain course completion certificates.

The project demonstrates the complete lifecycle of a cloud-native solution. The infrastructure was provisioned using Terraform, allowing all cloud resources to be created automatically through Infrastructure as Code. The application itself was containerized using Docker and deployed on an Azure Linux Virtual Machine. PostgreSQL was used as the database management system to store user accounts, course information, enrollments, and quiz results. In addition, a GitHub Actions CI/CD pipeline was implemented to automatically deploy application updates whenever new code is pushed to the repository.

The final solution demonstrates how modern DevOps practices can be applied to deploy and manage a production-style web application in the cloud. The project integrates cloud infrastructure provisioning, containerized application deployment, database management, backup and recovery procedures, and deployment automation into a single cohesive platform.



Figure 1: AIDA Home Page

2 Application Overview

AIDA (AI-driven Instructional Design Assistant) is a cloud-based educational platform designed to simplify the learning process through structured online courses and artificial intelligence-powered learning tools. The platform was developed to provide users with an accessible environment where they can study educational content, evaluate their knowledge, and track their learning progress through a modern web application.

The primary objective of AIDA is to combine traditional online learning with AI-assisted educational features. Instead of only providing static course materials, the platform allows learners to interact with dynamically generated quizzes and learning assessments. This approach helps users better understand course content while receiving immediate feedback regarding their performance.

The platform supports user registration and authentication, allowing learners to create personal accounts and securely access their educational data. After authentication, users can browse available courses, enroll in learning programs, complete lessons, and monitor their progress through a personalized dashboard. Course completion is tracked automatically, and users can generate certificates after successfully finishing a course.

One of the key features of AIDA is the integration of artificial intelligence tools. The platform includes an AI-powered quiz generation system that can analyze educational content and automatically create multiple-choice questions. In addition, users can upload PDF lecture materials and generate quizzes directly from uploaded documents. These AI capabilities provide a more interactive and personalized learning experience compared to traditional e-learning platforms.

The application follows a modern client-server architecture. The frontend is implemented using React and provides a responsive user interface optimized for both desktop and mobile devices. The backend is developed using FastAPI and exposes REST API endpoints that handle authentication, course management, lesson progression, AI functionality, and database operations. All persistent application data is stored in a PostgreSQL database.

The main features implemented in the AIDA platform include:

- User registration and authentication
- Course browsing and enrollment
- Lesson progression tracking
- Learning dashboard with progress statistics
- AI-generated quizzes from lesson content
- AI-generated quizzes from uploaded PDF files
- Quiz result storage and analysis
- Course completion certificates
- Responsive desktop and mobile user interface

The combination of educational content management, artificial intelligence services, and cloud-native deployment technologies makes AIDA a comprehensive example of a modern web-based learning platform. The application was selected for this project because it demonstrates the integration of multiple technologies including frontend development, backend services, database management, cloud infrastructure, containerization, and deployment automation.

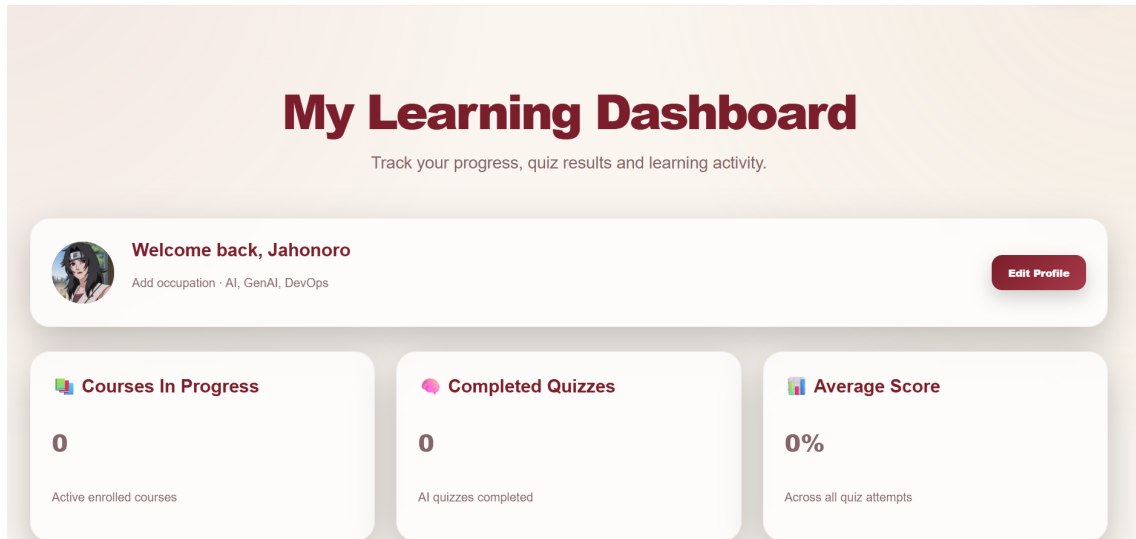


Figure 2: AIDA Dashboard

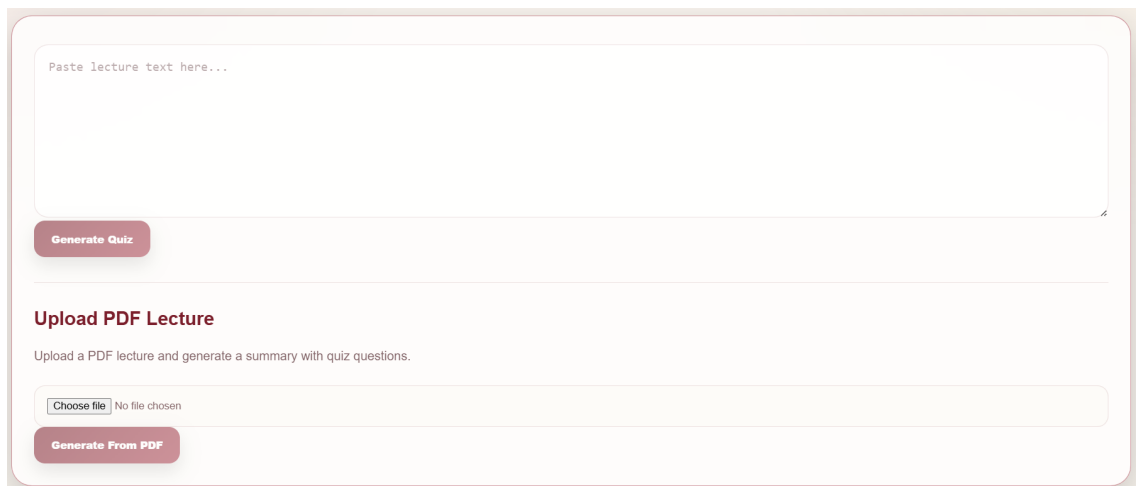


Figure 3: AIDA Learning Tools

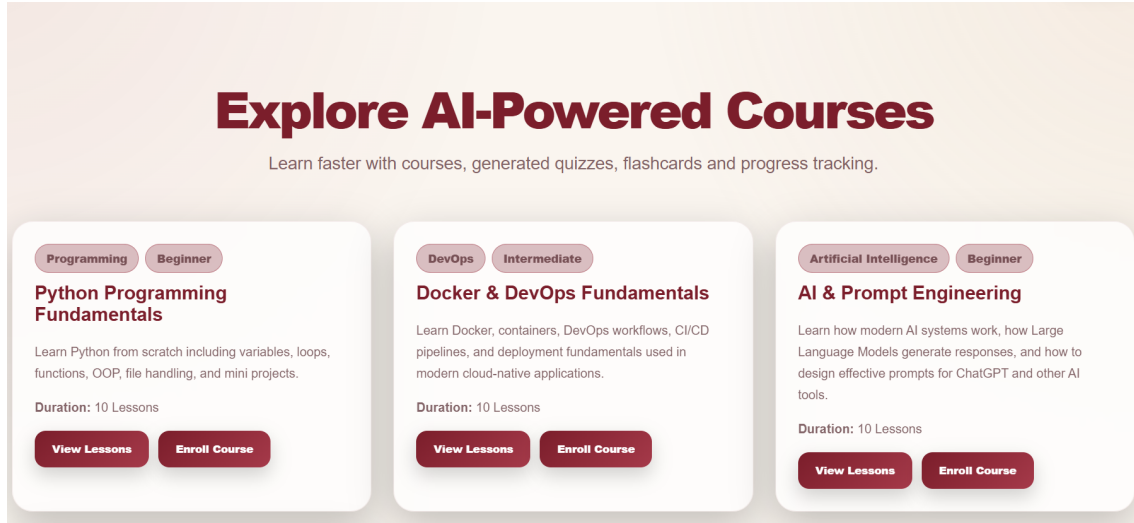


Figure 4: AIDA Course Details

3 System Architecture

The AIDA platform was designed using a modern three-tier architecture consisting of a frontend layer, a backend layer, and a database layer. This architectural approach was selected to ensure a clear separation of responsibilities between different components of the system, making the application easier to develop, maintain, and scale in the future. Each layer performs a specific role while communicating with the others through well-defined interfaces.

The frontend layer is implemented using React and serves as the primary interface between the user and the application. Through the web interface, users can register accounts, log in to the platform, browse available courses, enroll in learning programs, complete lessons, track their progress, and access AI-powered educational tools. React was chosen because it provides a responsive and dynamic user experience while allowing the application to be developed using reusable components. Special attention was given to responsive design to ensure that the platform remains accessible on both desktop and mobile devices.

The backend layer is implemented using FastAPI, a modern Python framework for building high-performance web APIs. The backend is responsible for processing requests received from the frontend, executing business logic, handling user authentication, managing course enrollments, tracking lesson completion, generating certificates, and integrating with artificial intelligence services. FastAPI was selected due to its excellent performance, automatic API documentation capabilities, and seamless integration with Python-based AI libraries and services.

Persistent application data is stored in a PostgreSQL database. The database contains information related to users, courses, lessons, enrollments, quiz attempts, and learning progress. PostgreSQL was chosen because it is a mature, reliable, and widely adopted relational database management system that provides strong data consistency and excellent performance for transactional workloads. By separating the database from the application logic, the platform ensures proper data management and simplifies future maintenance.

To support modern deployment practices, all application components are containerized using Docker. Individual containers are used for the React frontend, FastAPI backend, and PostgreSQL database. Docker Compose is responsible for orchestrating these services and managing communication between containers. This approach guarantees consistency between development and production environments while significantly simplifying deployment and maintenance procedures.

The complete application is deployed on a Linux Virtual Machine hosted in Microsoft Azure. Users access the frontend through a public IP address, while the frontend communicates with the backend through REST API requests. The backend then interacts with the PostgreSQL database whenever data needs to be stored, retrieved, or updated. This deployment model demonstrates a practical cloud-native architecture that combines modern web technologies, containerization,

database management, and cloud infrastructure into a single integrated solution.

Overall, the selected architecture provides a solid foundation for the AIDA platform. The separation of concerns between the frontend, backend, and database layers improves maintainability, while containerization and cloud deployment enable portability, reliability, and automated management through Infrastructure as Code and CI/CD practices.

4 Cloud Infrastructure Design

The cloud infrastructure for the AIDA platform was deployed using Microsoft Azure. The primary objective of the infrastructure design was to create a reliable environment capable of hosting the application while remaining simple enough to manage within the scope of the project. The infrastructure was provisioned entirely through Infrastructure as Code using Terraform, allowing all resources to be created and managed automatically.

The deployment is centered around a dedicated Azure Resource Group, which serves as a logical container for all cloud resources associated with the project. Using a Resource Group simplifies administration, monitoring, and resource lifecycle management because all components of the application can be managed as a single unit. In addition, it allows resources to be easily removed and recreated whenever infrastructure changes are required.

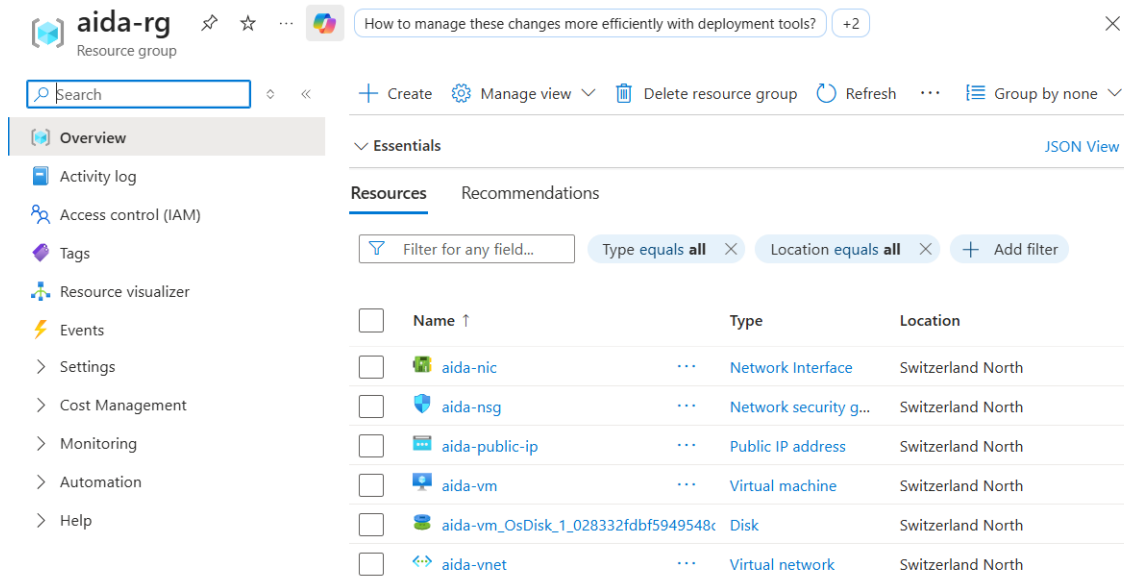


Figure 5: Azure Resource Group for the AIDA Platform

A Virtual Network (VNet) was created to provide network isolation and secure communication between infrastructure components. Within the virtual network, a dedicated subnet was configured to host the virtual machine running the application. This network structure follows Azure best practices by logically separating application resources from external networks while still allowing controlled public access.

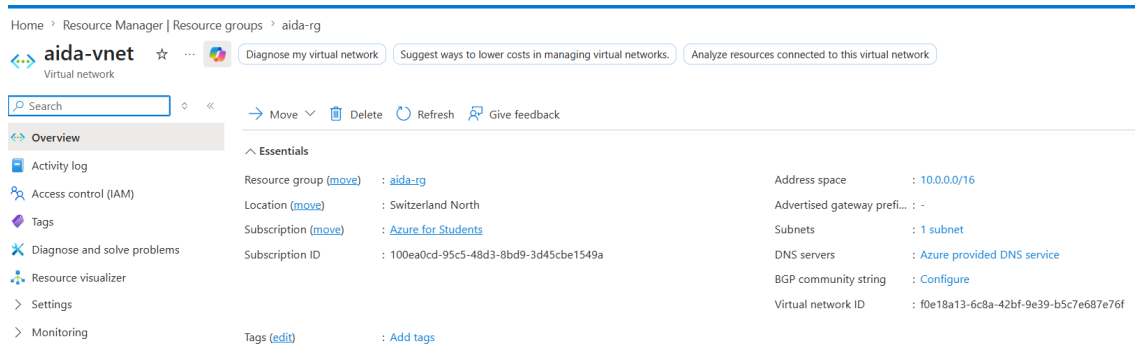


Figure 6: Azure Virtual Network and Subnet Configuration

To protect the application and control inbound traffic, a Network Security Group (NSG) was configured. The NSG contains a set of security rules that explicitly define which ports are accessible from the internet. Port 22 was opened to allow secure SSH access for administration purposes. Port 5173 was opened to provide access to the React frontend application, while port 8000 was configured to allow communication with the FastAPI backend service. All other traffic remains restricted according to Azure's default security model.

→ Move

🗑 Delete

🔄 Refresh

🗨 Give feedback

Essentials

JSON View

Resource group (move) : aida-rg

Custom security rules : 3 inbound, 0 outbound

Location : Switzerland North

Associated with : 0 subnets, 1 network interfaces

Subscription (move) : Azure for Students

Subscription ID : 100ea0cd-95c5-48d3-8bd9-3d45cbe1549a

Tags (edit) : Add tags

Filter by name

Port == all

Protocol == all

Source == all

Destination == all

Action == all

Priority	Name	Port	Protocol	Source	Destination	Action
Inbound Security Rules						
100	⚠ Allow-SSH	22	Tcp	Any	Any	✔ Allow
110	Allow-Frontend	5173	Tcp	Any	Any	✔ Allow
120	Allow-Backend	8000	Tcp	Any	Any	✔ Allow
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	✔ Allow
65001	AllowAzureLoadBalancer...	Any	Any	AzureLoadBalancer	Any	✔ Allow
65500	DenyAllInBound	Any	Any	Any	Any	✖ Deny
Outbound Security Rules						
65000	AllowVnetOutBound	Any	Any	VirtualNetwork	VirtualNetwork	✔ Allow
65001	AllowInternetOutBound	Any	Any	Any	Internet	✔ Allow
65500	DenyAllOutBound	Any	Any	Any	Any	✖ Deny

Figure 7: Network Security Group Configuration

A static public IP address was assigned to the infrastructure to ensure consistent external accessibility. This allows users to access the AIDA platform through a stable endpoint without requiring DNS reconfiguration after virtual machine restarts. The public IP address also serves as the connection point for remote administration and automated deployment processes.

Essentials			
Resource group (move)	: aida-rg	SKU	: Standard
Location (move)	: Switzerland North	Tier	: Regional
Subscription (move)	: Azure for Students	IP address	: 20.250.32.2
Subscription ID	: 100ea0cd-95c5-48d3-8bd9-3d45cbe1549a	DNS name	: -
		Associated to	: aida-nic
		Virtual machine	: aida-vm
		Domain name label scope	: -
		Routing preference	: Microsoft network
Tags (edit)	: Add tags		

Figure 8: Azure Public IP Configuration

The application itself is hosted on an Azure Linux Virtual Machine. Ubuntu Linux was selected as the operating system due to its stability, extensive community support, and strong compatibility with Docker-based workloads. The virtual machine hosts all application containers, including the frontend, backend, and PostgreSQL database. This deployment model provides a practical balance between simplicity, cost efficiency, and operational flexibility for a project of this scale.

Virtual machine

Help me copy this VM in any region

Manage this VM with Azure CLI

Search

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Monitor

Resource visualizer

Connect

Networking

Settings

Availability + scale

Security

Backup + disaster recovery

Operations

Monitoring

Automation

Help

Advisor (1 of 3): Enable Trusted Launch foundational excellence, and modern security for Existing Generation 2 VM(s)

Help me copy this VM in any region

Connect

Start

Restart

Stop

Hibernate

Capture

Delete

Essentials

Resource group ([move](#))

Status

Location

Subscription ([move](#))

Subscription ID

Operating system

Size

Primary NIC public IP

Virtual network/subnet

DNS name

Health state

Time created

Tags ([edit](#))

: [aida-rg](#)

: Running

: Switzerland North

: [Azure for Students](#)

: 100ea0cd-95c5-48d3-8bd9-3d45cbe154...

: Linux (ubuntu 24.04)

: Standard_B2ats_v2

: [20.250.32.2](#)

: [1 associated public IPs](#)

: [aida-vnet/aida-subnet](#)

: [Not configured](#)

: -

: 10/06/2026, 15:35 UTC

: [Add tags](#)

JSON View

Figure 9: Azure Linux Virtual Machine Hosting the Application

The chosen infrastructure architecture provides a reliable foundation for hosting the AIDA platform while supporting automation through Terraform and GitHub Actions. Although larger production systems often use managed services such as Azure App Service, Azure Kubernetes Service, or Azure Database for PostgreSQL, a virtual machine deployment was selected for this project because it offers full control over the environment and clearly demonstrates infrastructure

8

provisioning, application deployment, and operational management. This approach also aligns well with the educational objectives of the assignment by exposing all major infrastructure components and deployment processes.

5 Infrastructure as Code with Terraform

One of the primary requirements of this project was the implementation of Infrastructure as Code (IaC). To satisfy this requirement, Terraform was selected as the infrastructure provisioning tool. Terraform allows cloud resources to be defined using configuration files rather than being created manually through the Azure Portal. This approach improves reproducibility, reduces human error, and enables infrastructure to be version-controlled alongside application source code.

The entire cloud environment for the AIDA platform was described using Terraform configuration files. These files define all required Azure resources, including the Resource Group, Virtual Network, Subnet, Public IP Address, Network Security Group, and Linux Virtual Machine. By storing infrastructure definitions as code, the complete environment can be recreated consistently on demand.

Terraform communicates directly with Azure through the Azure Resource Manager (ARM) API. Authentication is performed using the Azure CLI, allowing Terraform to provision resources within the student's Azure subscription. Before deployment, Terraform validates the configuration and generates an execution plan showing the changes that will be applied. This provides an additional layer of safety by allowing infrastructure modifications to be reviewed before execution.

The deployment process begins with Terraform initialization. During this step, Terraform downloads the required Azure provider plugins and prepares the working directory. After initialization, the configuration is validated to ensure that no syntax or configuration errors exist. Terraform then generates an execution plan that displays the resources that will be created, modified, or destroyed. Finally, the infrastructure is provisioned using a single command.

The main deployment workflow used in this project is shown below:

```
terraform init
terraform validate
terraform plan
terraform apply
```

The use of Terraform significantly simplifies infrastructure management. Instead of manually creating networking resources, security rules, and virtual machines through graphical interfaces, the entire cloud environment can be provisioned automatically within a few minutes. This capability is particularly valuable in DevOps environments where consistency and repeatability are essential.

Terraform outputs were also configured to expose useful deployment information after infrastructure creation. These outputs include details such as the public IP address of the virtual machine and the virtual machine name itself. This information can be used immediately after deployment without requiring additional navigation through the Azure Portal.

```

PS C:\Users\jahon\Documents\study\main1_2\devOps\ai-course-generator\terraform> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# azurerm_linux_virtual_machine.aida_vm will be created
+ resource "azurerm_linux_virtual_machine" "aida_vm" {
  + admin_username           = "azureuser"
  + allow_extension_operations = (known after apply)
  + bypass_platform_safety_checks_on_user_schedule_enabled = false
  + computer_name            = (known after apply)
  + disable_password_authentication = (known after apply)
  + disk_controller_type     = (known after apply)
  + extensions_time_budget   = "PT1H30M"
  + id                      = (known after apply)
  + location                 = "switzerlandnorth"
  + max_bid_price            = -1
  + name                    = "aida-vm"
  + network_interface_ids    = (known after apply)
  + os_managed_disk_id       = (known after apply)
  + patch_assessment_mode    = (known after apply)
  + patch_mode               = (known after apply)
  + platform_fault_domain    = -1
  + priority                 = "Regular"
  + private_ip_address       = (known after apply)
  + private_ip_addresses     = (known after apply)
  + provision_vm_agent       = (known after apply)
  + public_ip_address        = (known after apply)
  + public_ip_addresses      = (known after apply)
  + resource_group_name      = "aida-rg"
  + size                     = "Standard_B2ats_v2"
  + virtual_machine_id       = (known after apply)
  + vm_agent_platform_updates_enabled = (known after apply)

  + admin_ssh_key {
    + public_key = <<-EOT
      ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIG+j1GN1hY3IXTW5QNu6EvH2+0w/9LcjlzlvCyT3AKBJH jahon@jahon6727
    }
  }

```

Figure 10: Terraform Execution Plan

```

# azurerm_subnet.aida_subnet will be created
+ resource "azurerm_subnet" "aida_subnet" {
  + address_prefixes = [
    + "10.0.1.0/24",
  ]
  + default_outbound_access_enabled = true
  + id                             = (known after apply)
  + name                           = "aida-subnet"
  + private_endpoint_network_policies = "Disabled"
  + private_link_service_network_policies_enabled = true
  + resource_group_name            = "aida-rg"
  + virtual_network_name           = "aida-vnet"
}

# azurerm_virtual_network.aida_vnet will be created
+ resource "azurerm_virtual_network" "aida_vnet" {
  + address_space = [
    + "10.0.0.0/16",
  ]
  + dns_servers = (known after apply)
  + guid        = (known after apply)
  + id          = (known after apply)
  + location    = "switzerlandnorth"
  + name        = "aida-vnet"
  + private_endpoint_vnet_policies = "Disabled"
  + resource_group_name            = "aida-rg"
  + subnet                        = (known after apply)
}

Plan: 8 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ public_ip_address = (known after apply)

```

Figure 11: Successful Terraform Infrastructure Deployment

```

PS C:\Users\jahon\Documents\study\main1_2\devOps\ai-course-generator\terraform> terraform output
public_ip_address = "20.250.32.2"
vm_name = "aida-vm"

```

Figure 12: Terraform Output Values

The adoption of Infrastructure as Code was one of the most important aspects of this project. It allowed the complete Azure environment to be created using a single deployment process while ensuring consistency between deployments. In addition, storing infrastructure definitions within the Git repository provides transparency, traceability, and version control for all infrastructure changes. This approach closely follows modern cloud engineering and DevOps best practices.

6 Containerization and Application Deployment

Containerization plays a central role in the deployment strategy of the AIDA platform. Instead of installing application dependencies directly on the operating system, each major component of the system is packaged inside an isolated Docker container. This approach ensures consistency between development and production environments while simplifying deployment, maintenance, and scalability.

The application consists of three primary services: the React frontend, the FastAPI backend, and the PostgreSQL database. Each service runs in its own dedicated container and communicates with the other components through Docker's internal networking capabilities. By isolating services into separate containers, individual components can be updated, restarted, or rebuilt without affecting the rest of the application.

The frontend service is responsible for presenting the user interface and handling interactions with the user. The React application is compiled into static production files and served through an Nginx web server running inside a Docker container. This provides efficient delivery of frontend assets while reducing resource consumption.

The backend service is implemented using FastAPI and is responsible for processing API requests, executing business logic, handling user authentication, generating AI-powered quizzes, managing courses, and interacting with the database. Running the backend inside a container ensures that all Python dependencies remain consistent across deployments and eliminates configuration differences between environments.

The PostgreSQL database is also deployed as a dedicated container. The database stores all persistent application data, including user accounts, course information, lessons, enrollments, quiz results, and learning progress. Containerizing the database simplifies deployment while still providing reliable data persistence through Docker volumes.

To manage all services simultaneously, Docker Compose was used. Docker Compose allows multiple containers to be defined within a single configuration file and started using a single command. This significantly simplifies deployment and operational management because the complete application stack can be launched, rebuilt, or stopped using a unified workflow.

The primary deployment command used throughout the project is shown below:

```
docker-compose up --build -d
```

This command automatically builds the frontend and backend images, creates the required containers, establishes networking between services, and starts the complete application environment in detached mode.

The running containers can be verified using the following command:

```
sudo docker ps
```

This command displays all active containers together with their status, exposed ports, and container identifiers. It serves as a useful operational tool for validating successful deployments and troubleshooting potential issues.

The containerized deployment model provides several important advantages. First, it guarantees consistent behavior across environments because all dependencies are packaged together with the application. Second, deployment becomes significantly faster because rebuilding and restarting containers requires minimal manual effort. Third, the architecture supports future scalability since services can be migrated to managed container platforms or Kubernetes environments with relatively minor modifications.

Containerization also contributed directly to the CI/CD implementation used in this project. Whenever new code is pushed to the repository, the deployment pipeline automatically rebuilds

the affected containers and deploys the updated application to the Azure Virtual Machine. This enables rapid delivery of new features while reducing deployment complexity and minimizing the possibility of configuration-related errors.

Overall, Docker and Docker Compose provided an effective solution for packaging, deploying, and managing the AIDA platform. The use of containerized services aligns closely with modern cloud-native development practices and supports reliable application operation within the Azure environment.

```
azureuser@aida-vm:~$ sudo docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
f986c248d5c6	ai-course-generator_frontend	ai_course_frontend	"/docker-entrypoint.s..."	About an hour ago	Up About an hour	0.0.0.0:51
73->80/tcp, [::]:5173->80/tcp						
7d71eaf48177	ai-course-generator_backend	ai_course_backend	"uvicorn app.main:ap..."	About an hour ago	Up About an hour	0.0.0.0:80
00->8000/tcp, [::]:8000->8000/tcp						
b601f6ca19d7	postgres:15	ai_course_db	"docker-entrypoint.s..."	About an hour ago	Up About an hour	0.0.0.0:54
33->5432/tcp, [::]:5433->5432/tcp						

Figure 13: Running Docker Containers on the Azure Virtual Machine

```
---> Using cache
---> 4702ba87697e
Step 4/6 : RUN pip install --no-cache-dir -r requirements.txt
---> Using cache
---> 061bbf8e6ddc
Step 5/6 : COPY ./app ./app
---> Using cache
---> 1efaea7918a8
Step 6/6 : CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
---> Using cache
---> 73fa40b9358a
Successfully built 73fa40b9358a
Successfully tagged ai-course-generator_backend:latest
Building frontend
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 2.729MB
Step 1/5 : FROM nginx:stable-alpine
---> 5f979dcfed4c
Step 2/5 : COPY dist /usr/share/nginx/html
---> Using cache
---> 6fe8d9377497
Step 3/5 : COPY nginx.conf /etc/nginx/conf.d/default.conf
---> Using cache
---> f43f82229c68
Step 4/5 : EXPOSE 80
---> Using cache
---> 2dbd0333efaa
Step 5/5 : CMD ["nginx", "-g", "daemon off;"]
---> Using cache
---> 3a48643b4d95
Successfully built 3a48643b4d95
Successfully tagged ai-course-generator_frontend:latest
ai_course_db is up-to-date
ai_course_backend is up-to-date
ai_course_frontend is up-to-date
azureuser@aida-vm:~/ai-course-generator$
```

Figure 14: Docker Compose Deployment Process

7 Database Backup and Recovery

Reliable data storage is an important requirement for any production-style application. Since the AIDA platform stores user accounts, courses, lesson progress, and quiz results, it was necessary to implement a backup and recovery strategy to prevent data loss.

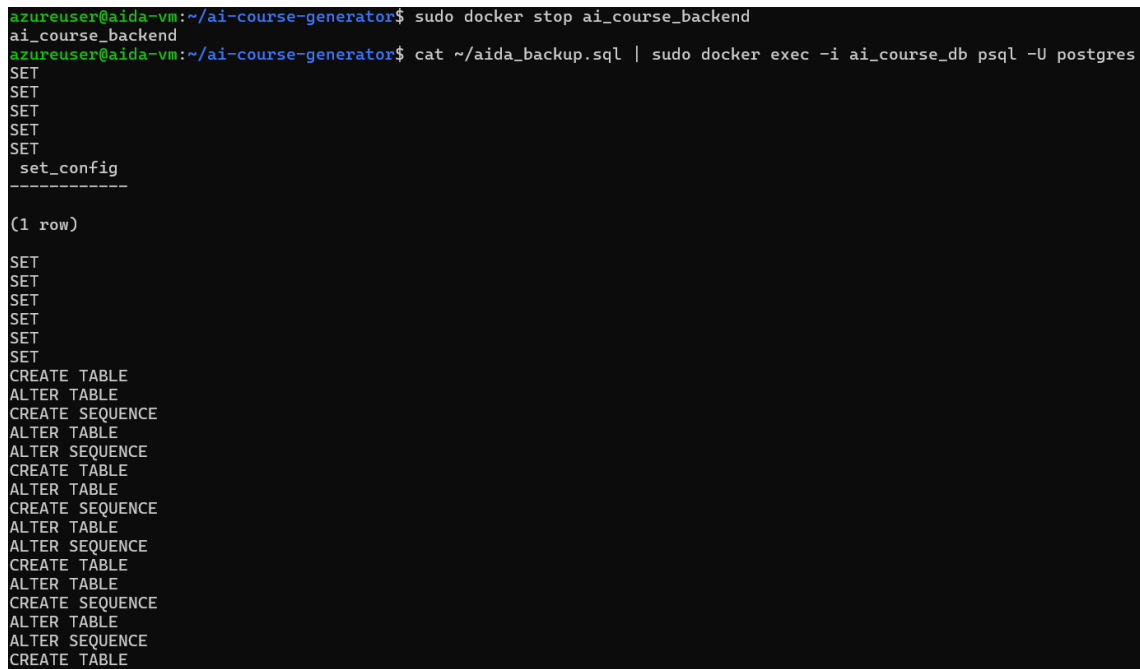
Database backups were created using PostgreSQL's built-in pg dump utility. This tool generates a complete export of the database, including both the schema and stored data. The backup file can then be transferred to another server and restored whenever necessary.

The following command was used to create a database backup:

```
docker exec ai_course_db pg_dump -U postgres
-d ai_course_db > aida_backup.sql
```

The backup was later successfully restored on a newly deployed Azure Virtual Machine after the infrastructure was recreated. This recovery process restored all application data, including registered users and course information, demonstrating that the platform can recover from infrastructure replacement without permanent data loss.

Implementing a backup and recovery procedure increases the reliability of the system and ensures that important educational data can be preserved and restored when required.



```
azureuser@aida-vm:~/ai-course-generator$ sudo docker stop ai_course_backend
ai_course_backend
azureuser@aida-vm:~/ai-course-generator$ cat ~/aida_backup.sql | sudo docker exec -i ai_course_db psql -U postgres
SET
SET
SET
SET
SET
SET
  set_config
-----
(1 row)
SET
SET
SET
SET
SET
SET
SET
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER TABLE
ALTER SEQUENCE
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER TABLE
ALTER SEQUENCE
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER TABLE
ALTER SEQUENCE
CREATE TABLE
ALTER TABLE
```

Figure 15: PostgreSQL Database Backup Process

8 CI/CD Pipeline Implementation

Continuous Integration and Continuous Deployment (CI/CD) was implemented to automate the deployment process of the AIDA platform. The primary goal of the pipeline was to ensure that application updates could be deployed automatically whenever changes were pushed to the GitHub repository. This approach eliminates manual deployment steps, reduces the possibility of human error, and enables faster delivery of new features and bug fixes.

GitHub Actions was selected as the CI/CD platform because it integrates directly with GitHub repositories and provides a simple yet powerful way to automate software delivery workflows. The deployment pipeline is triggered automatically whenever code is pushed to the `main` branch of the repository. As a result, every approved code change is automatically delivered to the production environment running on Microsoft Azure.

The deployment workflow is defined in a GitHub Actions configuration file located in the repository. When the workflow is triggered, GitHub Actions establishes a secure SSH connection to the Azure Virtual Machine using a private deployment key stored as a GitHub Secret. This ensures that deployment credentials remain protected and are not exposed within the repository.

After successfully connecting to the virtual machine, the workflow automatically performs several deployment tasks. First, the latest source code is downloaded from the GitHub repository using a Git pull operation. Next, the frontend application dependencies are installed and the React application is rebuilt for production. Once the build process is completed, Docker Compose is used to stop the currently running containers, rebuild application images if necessary, and start the updated containers in the background.

The deployment workflow used in this project can be summarized as follows:

```
Developer Pushes Code
|
v
GitHub Repository
|
v
GitHub Actions
|
v
SSH Connection
|
v
Azure VM
|
v
Docker Compose Rebuild
|
v
Updated Application
```

One of the major advantages of this approach is that no manual deployment actions are required after code changes are committed. The entire process, from source code update to application deployment, is executed automatically. This closely follows modern DevOps practices where software delivery is automated and infrastructure changes can be applied rapidly and consistently.

To secure the deployment process, sensitive information such as the server IP address, username, and SSH private key were stored as encrypted GitHub Secrets. This prevents confidential deployment credentials from being exposed in source code while still allowing the workflow to authenticate securely with the production server.

The implemented CI/CD pipeline significantly improved the operational workflow of the project. During development, new features, UI changes, and backend updates could be deployed to the Azure environment simply by committing and pushing code to the repository. GitHub Actions automatically handled the remaining deployment steps, allowing development efforts to focus on application functionality rather than manual infrastructure management.

The successful implementation of automated deployment demonstrates how modern cloud-native applications can benefit from Continuous Integration and Continuous Deployment practices. The solution satisfies the project requirement that code changes pushed to the repository automatically trigger a deployment to the cloud environment without requiring manual intervention.

All workflows

Showing runs from all workflows

Filter workflow runs

6 workflow runs		Workflow	Event	Status	Branch	Actor
✓	test CD/CI	Deploy AIDA to Azure VM #6: Commit ebc336b pushed by Jahon27	main	Jun 10, 23:56 GMT+2	28s	...
✓	testing 4 update	Deploy AIDA to Azure VM #5: Commit c64925e pushed by Jahon27	main	Jun 10, 20:17 GMT+2	30s	...
✓	testing update	Deploy AIDA to Azure VM #4: Commit 5b59bf0 pushed by Jahon27	main	Jun 10, 20:14 GMT+2	30s	...
✓	testing update	Deploy AIDA to Azure VM #3: Commit 8eb0c58 pushed by Jahon27	main	Jun 10, 20:13 GMT+2	30s	...
✓	testing update	Deploy AIDA to Azure VM #2: Commit 762c591 pushed by Jahon27	main	Jun 10, 20:12 GMT+2	37s	...
✓	Add CI/CD deployment pipeline	Deploy AIDA to Azure VM #1: Commit fc45d6b pushed by Jahon27	main	Jun 10, 19:48 GMT+2	22s	...

Figure 16: Successful GitHub Actions Deployment Workflow

Repository secrets

New repository secret

Name	Last updated
AZURE_HOST	5 hours ago
AZURE_SSH_KEY	5 hours ago
AZURE_USER	5 hours ago

Figure 17: GitHub Repository Secrets Used for Deployment


```
✓ Build appleboy/ssh-action@v1.0.3

1 ▼ Build container for action use: '/home/runner/work/_actions/appleboy/ssh-action/v1.0.3/Dockerfile'.
2 /usr/bin/docker build -t eee315:26d152f4db5d4a7cb97f6f040ea42cbc -f "/home/runner/work/_actions/appleboy/ssh-action/v1.0.3/Dockerfile"
   "/home/runner/work/_actions/appleboy/ssh-action/v1.0.3"
3 #0 building with "default" instance using docker driver
4
5 #1 [internal] load build definition from Dockerfile
6 #1 transferring dockerfile: 149B 0.0s done
7 #1 DONE 0.0s
8
9 #2 [internal] load metadata for ghcr.io/appleboy/drone-ssh:1.7.3
10 #2 DONE 0.9s
11
12 #3 [internal] load .dockerignore
13 #3 transferring context: 2B done
14 #3 DONE 0.0s
15
16 #4 [internal] load build context
17 #4 transferring context: 108B done
18 #4 DONE 0.0s
19
20 #5 [1/2] FROM ghcr.io/appleboy/drone-ssh:1.7.3@sha256:179cbd253771007f2ebcaf7a3860afe059207351de1f3378efafac93bd310158
21 #5 resolve ghcr.io/appleboy/drone-ssh:1.7.3@sha256:179cbd253771007f2ebcaf7a3860afe059207351de1f3378efafac93bd310158 done
22 #5 sha256:179cbd253771007f2ebcaf7a3860afe059207351de1f3378efafac93bd310158 2.38kB / 2.38kB done
23 #5 sha256:1a52f94820e830bc9734623deb03bbafa6a248fba19cdc2e5605c70d90d914141 1.24kB / 1.24kB done
24 #5 sha256:c4781b8d012b16b6c7468dc9104fad0a63f2a8815b2af8b7176bb544253ea8c7 3.86kB / 3.86kB done
25 #5 sha256:1207c741d8c9b028d98c4006013f9de959da3c55f85b91ed5e4583438a0112ca 0B / 3.38MB 0.1s
26 #5 sha256:654aaee5654be60f81172539ecd74485cf8bbe2e9137bab0d429862c31c676f6 0B / 284.83kB 0.1s
27 #5 sha256:7e20f48a19643465f486f59127b8f4e0f4114bedc8bd5260fb4f92d5fda95f1c 0B / 1.20kB 0.1s
28 #5 sha256:7e20f48a19643465f486f59127b8f4e0f4114bedc8bd5260fb4f92d5fda95f1c 1.20kB / 1.20kB 0.1s done
29 #5 sha256:066e5b821e26d8313055e0314c056e6c297847bcf963e5df10e7aa03107f2246 0B / 125B 0.2s
30 #5 sha256:1207c741d8c9b028d98c4006013f9de959da3c55f85b91ed5e4583438a0112ca 3.38MB / 3.38MB 0.3s done
31 #5 sha256:654aaee5654be60f81172539ecd74485cf8bbe2e9137bab0d429862c31c676f6 284.83kB / 284.83kB 0.2s done
32 #5 sha256:066e5b821e26d8313055e0314c056e6c297847bcf963e5df10e7aa03107f2246 125B / 125B 0.2s done
33 #5 extracting sha256:1207c741d8c9b028d98c4006013f9de959da3c55f85b91ed5e4583438a0112ca 0.1s done
```

Figure 18: Deployment Workflow Execution Log

```
✓ Deploy via SSH 14s

1 ▶ Run appleboy/ssh-action@v1.0.3
20 /usr/bin/docker run --name eee31526d152f4db5d4a7cb97f6f040ea42cbc_4d6db0 --label eee315 --workdir /github/workspace --rm -e "INPUT_HOST" -e "INPUT_USERNAME"
   -e "INPUT_KEY" -e "INPUT_SCRIPT" -e "INPUT_PORT" -e "INPUT_PASSPHRASE" -e "INPUT_PASSWORD" -e "INPUT_SYNC" -e "INPUT_USE_INSECURE_CIPHER" -e "INPUT_CIPHER"
   -e "INPUT_TIMEOUT" -e "INPUT_COMMAND_TIMEOUT" -e "INPUT_KEY_PATH" -e "INPUT_FINGERPRINT" -e "INPUT_PROXY_HOST" -e "INPUT_PROXY_PORT" -e "INPUT_PROXY_USERNAME"
   -e "INPUT_PROXY_PASSWORD" -e "INPUT_PROXY_PASSPHRASE" -e "INPUT_PROXY_TIMEOUT" -e "INPUT_PROXY_KEY" -e "INPUT_PROXY_KEY_PATH" -e "INPUT_PROXY_FINGERPRINT" -e
   "INPUT_PROXY_CIPHER" -e "INPUT_PROXY_USE_INSECURE_CIPHER" -e "INPUT_SCRIPT_STOP" -e "INPUT_ENVS" -e "INPUT_ENVS_FORMAT" -e "INPUT_DEBUG" -e "INPUT_ALLENVS"
   -e "INPUT_REQUEST_PTY" -e "HOME" -e "GITHUB_JOB" -e "GITHUB_REF" -e "GITHUB_SHA" -e "GITHUB_REPOSITORY" -e "GITHUB_REPOSITORY_OWNER" -e
   "GITHUB_REPOSITORY_OWNER_ID" -e "GITHUB_RUN_ID" -e "GITHUB_RUN_NUMBER" -e "GITHUB_RETENTION_DAYS" -e "GITHUB_RUN_ATTEMPT" -e "GITHUB_ACTOR_ID" -e
   "GITHUB_ACTOR" -e "GITHUB_WORKFLOW" -e "GITHUB_HEAD_REF" -e "GITHUB_BASE_REF" -e "GITHUB_EVENT_NAME" -e "GITHUB_SERVER_URL" -e "GITHUB_API_URL" -e
   "GITHUB_GRAPHQL_URL" -e "GITHUB_REF_NAME" -e "GITHUB_REF_PROTECTED" -e "GITHUB_REF_TYPE" -e "GITHUB_WORKFLOW_REF" -e "GITHUB_WORKFLOW_SHA" -e
   "GITHUB_REPOSITORY_ID" -e "GITHUB_TRIGGERING_ACTOR" -e "GITHUB_WORKSPACE" -e "GITHUB_ACTION" -e "GITHUB_EVENT_PATH" -e "GITHUB_ACTION_REPOSITORY" -e
   "GITHUB_ACTION_REF" -e "GITHUB_PATH" -e "GITHUB_ENV" -e "GITHUB_STEP_SUMMARY" -e "GITHUB_STATE" -e "GITHUB_OUTPUT" -e "RUNNER_OS" -e "RUNNER_ARCH" -e
   "RUNNER_NAME" -e "RUNNER_ENVIRONMENT" -e "RUNNER_TOOL_CACHE" -e "RUNNER_TEMP" -e "RUNNER_WORKSPACE" -e "ACTIONS_RUNTIME_URL" -e "ACTIONS_RUNTIME_TOKEN" -e
   "ACTIONS_CACHE_URL" -e "ACTIONS_RESULTS_URL" -e "ACTIONS_ORCHESTRATION_ID" -e GITHUB_ACTIONS=true -e CI=true -v "/var/run/docker.sock":"/var/run/docker.sock"
   -v "/home/runner/work/_temp":"/github/runner_temp" -v "/home/runner/work/_temp/_github_home":"/github/home" -v
   "/home/runner/work/_temp/_github_workflow":"/github/workflow" -v "/home/runner/work/_temp/_runner_file_commands":"/github/file_commands" -v
   "/home/runner/work/ai-course-generator/ai-course-generator":"/github/workspace" eee315:26d152f4db5d4a7cb97f6f040ea42cbc
21 =====CMD=====
22 cd ~/ai-course-generator
23 git pull origin main
24 cd frontend
25 npm install
26 npm run build
27 cd ..
28 sudo docker-compose down
29 sudo docker-compose up --build -d
30 sudo docker ps
31 =====END=====
32 err: From https://github.com/Jahon27/ai-course-generator
33 err: * branch          main          -> FETCH_HEAD
34 err: c64925e..ebc336b main          -> origin/main
35 out: Updating c64925e..ebc336b
36 out: Fast-forward
37 out: frontend/src/App.css | 2 +--
```

Figure 19: Deployment Workflow Execution Log

9 Security Considerations

Although the primary focus of this project was cloud deployment and automation, several security measures were implemented to protect the application and its infrastructure.

User authentication is handled by the FastAPI backend, ensuring that only registered users can access protected functionality such as course enrollment, lesson progression, and AI-powered learning tools. Authentication tokens are used to verify user identity and restrict unauthorized access to application resources.

At the infrastructure level, Azure Network Security Groups (NSGs) were configured to control inbound traffic to the virtual machine. Only the required ports were opened, including SSH for administration and the application ports used by the frontend and backend services. This reduces the attack surface by limiting unnecessary network exposure.

For the CI/CD pipeline, sensitive deployment credentials were not stored in the source code repository. Instead, GitHub Secrets were used to securely store the server IP address, username, and SSH private key required for automated deployments. This ensures that confidential information remains protected while still allowing GitHub Actions to perform deployments automatically.

In addition, regular database backups were created to protect against accidental data loss and to support disaster recovery. This provides an additional layer of operational security by ensuring that important application data can be restored if required.

While this project does not implement advanced enterprise-level security features, the selected controls provide an appropriate level of protection for a cloud-hosted educational application and follow common security practices used in modern web development.

10 Challenges and Solutions

During the development and deployment of the AIDA platform, several challenges were encountered. These challenges provided valuable practical experience in cloud computing, DevOps practices, and infrastructure management.

One of the first challenges was the development of the application itself. Building a complete educational platform required the integration of multiple technologies, including React for the frontend, FastAPI for the backend, PostgreSQL for data storage, and Docker for containerization. A significant amount of time was spent designing the application structure, implementing user authentication, creating course and lesson functionality, and integrating AI-powered quiz generation features. As the project evolved, additional effort was required to ensure that all components communicated correctly and that the user experience remained intuitive.

Another major challenge occurred during the migration of the application to the cloud environment. While recreating infrastructure using Terraform, a new virtual machine was provisioned, which initially resulted in the loss of previously stored database content. At that point, the application itself was functioning correctly, but all users, courses, and learning progress data were missing because the database had not yet been restored.

To solve this problem, a PostgreSQL backup strategy was implemented using the pg_dump utility. A complete backup of the database was created before infrastructure changes were made. After the new virtual machine was deployed, the backup was successfully restored, allowing all users, courses, and application data to be recovered. This experience highlighted the importance of backup and disaster recovery procedures in real-world cloud environments.

Another challenge involved configuring the CI/CD pipeline using GitHub Actions. Establishing secure SSH-based communication between GitHub and the Azure Virtual Machine required proper key management and workflow configuration. Several deployment issues were encountered during the initial setup, including authentication errors and deployment permissions. These issues were resolved by generating dedicated deployment keys, storing credentials securely using GitHub Secrets, and validating the deployment workflow through multiple test deployments.

Overall, the challenges encountered throughout the project significantly improved practical understanding of cloud infrastructure, automated deployments, containerized applications, and database administration. Solving these problems contributed directly to the successful completion of the final cloud-native solution.

11 Challenges and Solutions

During the development and deployment of the AIDA platform, several challenges were encountered. These challenges provided valuable practical experience in cloud computing, DevOps practices, and infrastructure management.

One of the first challenges was the development of the application itself. Building a complete educational platform required the integration of multiple technologies, including React for the frontend, FastAPI for the backend, PostgreSQL for data storage, and Docker for containerization. A significant amount of time was spent designing the application structure, implementing user authentication, creating course and lesson functionality, and integrating AI-powered quiz generation features. As the project evolved, additional effort was required to ensure that all components communicated correctly and that the user experience remained intuitive.

Another major challenge occurred during the migration of the application to the cloud environment. While recreating infrastructure using Terraform, a new virtual machine was provisioned, which initially resulted in the loss of previously stored database content. At that point, the application itself was functioning correctly, but all users, courses, and learning progress data were missing because the database had not yet been restored.

To solve this problem, a PostgreSQL backup strategy was implemented using the pg_dump utility. A complete backup of the database was created before infrastructure changes were made. After the new virtual machine was deployed, the backup was successfully restored, allowing all users, courses, and application data to be recovered. This experience highlighted the importance of backup and disaster recovery procedures in real-world cloud environments.

Another challenge involved configuring the CI/CD pipeline using GitHub Actions. Establishing secure SSH-based communication between GitHub and the Azure Virtual Machine required proper key management and workflow configuration. Several deployment issues were encountered during the initial setup, including authentication errors and deployment permissions. These issues were resolved by generating dedicated deployment keys, storing credentials securely using GitHub Secrets, and validating the deployment workflow through multiple test deployments.

Overall, the challenges encountered throughout the project significantly improved practical understanding of cloud infrastructure, automated deployments, containerized applications, and database administration. Solving these problems contributed directly to the successful completion of the final cloud-native solution.